

Numerical Solution Error and Taylor Series

You must complete all exercises and submit as PDF

Errors in Numerical Solutions

Numerical method is processed in a digital system such as PC or MCU. The main problem of numerical method is that it can produce errors due to limited number of digits representing numbers, chopping, round-off, and truncation errors. We will analyze the total error of the numerical solution.

Representation of Error

Absolute total error:

$$\text{Absolute Error} = |\text{True Value} - \text{Numerical Solution}|$$

No. For example,

A.E. of 10mm $\rightarrow |10\text{mm} - 11\text{mm}| = 1$
(absolute error)

A.E. of 1000mm $\rightarrow |1000\text{mm} - 1001\text{mm}| = 1$

The absolute errors of them are the same.

We need to be careful how to interpret the absolute error. For example, assume that we have obtained the solution with the true error of 1mm. Can we say this is an accurate solution?

> Compare the absolute error of '1mm' from measuring a length of 10mm vs 1000mm

However, the relative error of 10mm

is $|\frac{1}{10}| = 0.1$, while that

of 1000mm is $|\frac{1}{1000}| = 0.001$.

Relative total error

$$\text{True Relative Error} = \left| \frac{\text{True} - \text{N.S.}}{\text{True}} \right|$$

This is non-dimensional and scale-independent.

In general, the true value is not known when solving numerical problems. Others means are used to evaluate the error or the accuracy, such as estimating the error bounds and approximating the order of magnitude of errors.

Accuracy: how close a given set of measurements are to their true value.
Precision: how close the measurements are to each other.

Accuracy vs Precision

It is important to note the accuracy to n decimal places or significant digits in measuring or recording a number. What is the difference between the accuracy and precision?

Example:

- A sensor measures a distance with 10mm accuracy and 1mm of precision (1σ)?

Yes, since you will consistently get the similar values that are close to one another and to the true value 10mm.

Significant figures(digits)

$$x = 1.2 + 3.45 \Rightarrow 1.2 \rightarrow 2 \text{ sig.}, 3.45 \rightarrow 3 \text{ sig.}$$

$$1.2 \Rightarrow 1.15 \sim 1.249$$

$$3.45 \Rightarrow 3.445 \sim 3.454$$

$$\Leftrightarrow 4.595 \leq x \leq 4.703$$

so, x has only 2 significant digits

Total Numerical Error

The numerical methods use approximations for solving problems. Also, in general, numerical solutions are not exact. The total error of the numerical solution is a combination of various errors. Also, there can be measurement errors in the data especially from the experimental data. We cannot get the exact value of the total error if we do not know the true value.

$$\text{Total error} = \text{Truncation error} + \text{Round-off error}$$

The total error of the numerical solution is a combination of various errors. The total error is consisted of

- Round-off error**
- Truncation error**

Rounding-off Error(Loss of Significance)

Numbers are represented in digital systems with a fixed number of bits, such as 16-bits, 32-bits MCU, 64-bit CPU. Due to the finite number of bits in representing numbers, there exists **Round-off Errors** in numerical solutions.

For example, the number $2/3$ cannot be represented exactly in a computer and it needs to be **chopped-off** and the last digits requires to be **rounded**.

This error could be small, but it can be accumulated over the calculation process.

Window 64=bit (see APPENDIX A)

	Bit	Byte	Numbers
Integer	32	4	-2,147,483,648~ 2,147,483,647
Unsigned int	32	4	0~ 4,294,967,295.
float	32	4	$\pm 3.4e-38$ ($\pm 3.4e-38$)
double	64	8	$\pm 1.8e308$ ($\pm 1.1e-308$)

Problems with Finite bits in Digital System

Finite number of bits to represent numbers causes

- Underflow: the smallest number closest to zero. Cannot define number in between.
- Overflow : has upper(+)/lower(-) limits on the number representation.
- There is a gap between two consecutive smallest numbers.

Thus, numbers can be rounded-off or chopped

Chopping

Due to number of digits that can be represented, digits are cut-off without rounding.

$\rho = 9.8909 \times 10^3$, $\rightarrow$ if the CPU/Compiler can have 3 Mantissa,
then $\rho = 9.890 \times 10^3$

Rounding

Due to number of digits that can be represented, cut off digits with rounding

$$\rho = 9.8909 \times 10^3 \rightarrow \rho = 9.891 \times 10^3 (4 \text{ s.f})$$

Loss of significance due to rounding-off error

When two numbers that are nearly the same are subtracted, we may get significant errors

$$\text{when } x \approx y \text{ or } \left|1 - \frac{y}{x}\right| \approx 0 \text{ for subtraction of } (x - y)$$

Theorem on loss of precision:

At most ***p*** & at least ***q*** significant binary bit are lost in computing $x - y$, , $q > p$.

$$2^{-p} \leq 1 - \frac{y}{x} \leq 2^{-q},$$

This is losing number of digits to represent a number!

Example

$$y = 0.6311, x = 0.6353$$

$$x - y = 4.2 \times 10^{-3} \text{ (2 sig. digits)}$$

Lost 2 significant digits. It is accurate to 2 significant digits

↗ You need to do a root rationalization to prevent the error from converging to zero.

How can we reduce loss of significance or rounded-off errors for subtracting two nearly the same numbers?

- Example: Solve a quadratic equation.

$$x^2 - 100.0001x + 0.001 = 0$$

True solutions are : $x_1=100$, $x_2=0.0001$ from

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} = \frac{100.0001 \pm (99.999)}{2} \quad . \text{ Note } -b \approx \sqrt{b^2 - 4ac} \text{ for this case}$$

If the roots are solved in a digital ssyte,
for x_1 ,

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = 100$$

for x_2

$$x = \frac{-b - \sqrt{b^2 - 4ac}}{2a} = 1.00...033 \times 10^{-4} \quad (\text{not exact value})$$

If expression is rearranged as below, we can get an exact value

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \rightarrow x_2 = \frac{2c}{-b + \sqrt{b^2 - 4ac}} = 0.0001$$

Exercise : Solve $f(x) = x(\sqrt{x} - \sqrt{x-1})$, when $x=100,000$. Assume the computer has only 6 significant digits.

True value $\Rightarrow f(100,000) = 158.1142$

Option 1)

$$f(x) = x(\sqrt{x} - \sqrt{x-1})$$

$$(1) \Rightarrow f(100,000) = 100,000\sqrt{100,000} - \sqrt{100,000-1} = 100,000(316.228 - 316.226) = 200.$$

Option 2)

Rearrange to avoid near-number subtraction. Then solve for $f(100,000)$. Is this answer closer to the true value?

$$f(x) = \frac{x(\sqrt{x} - \sqrt{x-1})(\sqrt{x} + \sqrt{x-1})}{(\sqrt{x} + \sqrt{x-1})} = \frac{x(\cancel{\sqrt{x}} - (\cancel{\sqrt{x}} - 1))}{\sqrt{x} + \sqrt{x-1}}$$

$$= \frac{x}{\sqrt{x} + \sqrt{x-1}}$$

$$f(100,000) = \frac{100,000}{\sqrt{100,000} + \sqrt{100,000-1}} = 158.1142783 \dots$$

much closer to the true value.

Truncation Error

It occurs when the numerical method uses approximate mathematical procedures.

Examples.

1) Taylor series

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

→ Since we cannot calculate up to $k = \infty$, there is truncation error.

2) Derivative approximation

$$\left. \frac{df}{dx} \right|_{x=x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1} + \varepsilon$$

for (int k=0; k<N; k++) {

y += pow(-1, k) * pow(x, 2k+1) /
factorial(2k+1);

Exercise

How can we program a numerical method for $\sin(x)$? }

Hint: use Taylor series and assume $x_0=0$

Use Taylor's expansion to express $\sin(x)$

$$\text{Taylor's expansion} = f(x_0) + f'(x_0)(x-x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2 + \frac{f'''(x_0)}{3!}(x-x_0)^3 + \dots$$

$$f(x) = \sin x$$

$$f(x_0) = \sin 0 = 0$$

$$f'(x_0) = \cos 0 = 1$$

$$f''(x_0) = -\sin 0 = 0$$

$$f'''(x_0) = -\cos 0 = -1$$

$$f^{(4)}(x_0) = \sin 0 = 0$$

;

$$\sin x = 0 + 1 \cdot (x-0) + \frac{0}{2!}(x-0)^2 + \frac{-1}{3!}(x-0)^3$$

$$+ \frac{0}{4!}(x-0)^4 + \frac{1}{5!}(x-0)^5 + \dots$$

$$= x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 - \frac{1}{7!}x^7 + \frac{1}{9!}x^9 + \dots$$

$$\therefore \boxed{\sin x = \sum_{k=0}^{\infty} (-1)^k \cdot \frac{x^{(2k+1)}}{(2k+1)!}}$$

Taylor Series

It is a powerful method of representing a function (x) with a sum of infinite convergent series of a simpler function would make calculation much simpler.

It can be used to find the value of a function near a known point(x_0). Taylor series converts rapidly near the point of expansion (i.e. $x=0$ for $\sin(x)$) and slowly at more remote points.

Accuracy increases as the number of terms increases and as x is closer to x_0

It is useful to represent a function such as sine, cosine, in a program for numerical method. Normally, we use 2~3 terms for approximation

Taylor Series Expansion

Taylor Series for f about point x_0

Assumption: $f(x)$ is differentiable $(n+1)$ times in an interval that contains x_0

Theorem 1:

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{(x - x_0)^2}{2!} f''(x_0) + \cdots + \frac{(x - x_0)^n}{n!} f^{(n)}(x_0) + \dots$$

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k$$

Theorem 2: $I = [a, b]$, $a \leq x_0 \leq b$, $f(x)$ differentiable $(n+1)$ times

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{(x - x_0)^2}{2!} f''(x_0) + \cdots + \frac{(x - x_0)^n}{n!} f^{(n)}(x_0) + E_{n+1}$$

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + E_{n+1}$$

$$\text{where, } E_{n+1} = f^{(n+1)}(\xi) \frac{(x - x_0)^{n+1}}{(n+1)!}, \quad \xi \text{ is between } [x, x_0]$$

Truncation error

Exercise: Express each functions with Taylor series

(1) e^x , at $x_0 = 0$, (2) $\sin(x)$, at $x_0 = 0$, (3) $\cos(x)$, at $x_0 = 0$

(1) $f(x) = e^x$
 $f(x_0) = e^0 = 1$
 $f'(x_0) = e^0 = 1$

$$\therefore e^x = \sum_{k=0}^{\infty} \frac{1}{k!} x^k$$

$\vdots$
 $e^x = \frac{f(x_0)}{0!} + \frac{f'(x_0)}{1!} (x-x_0) + \frac{f''(x_0)}{2!} (x-x_0)^2 + \dots$
 $= 1 + x + \frac{1}{2!} x^2 + \frac{1}{3!} x^3 + \dots$

(2) $f(x) = \sin x$

$$\sin x = 0 + x + 0 - \frac{1}{3!} x^3 + 0 + \frac{1}{5!} x^5 + \dots$$

$f(x_0) = \sin 0 = 0$

$f'(x_0) = \cos 0 = 1$

$f''(x_0) = -\sin 0 = 0$

$f^3(x_0) = -\cos 0 = -1$

$f^4(x_0) = \sin 0 = 0$

$\vdots$

$$\therefore \sin x = \sum_{k=0}^{\infty} (-1)^k \cdot \frac{x^{(2k+1)}}{(2k+1)!}$$

(3) $f(x) = \cos x$

$$\cos x = 1 + 0 + \frac{-1}{2!} x^2 + 0 + \frac{1}{4!} x^4 + 0$$

$f(x_0) = \cos 0 = 1$

$f'(x_0) = -\sin 0 = 0$

$f''(x_0) = -\cos 0 = -1$

$f^3(x_0) = \sin 0 = 0$

$f^4(x_0) = \cos 0 = 1$

$\vdots$

$$\therefore \cos x = \sum_{k=0}^{\infty} (-1)^k \cdot \frac{1}{(2k)!} \cdot x^{2k}$$

$$f(x_0) + f'(x_0)(x-x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2$$

How can we find the value of at $f(x + \Delta x) = ?$

Taylor Series for $f(x+h)$

Let $\Delta x = h$ and $x_0 \rightarrow x$ is the expansion point

$x + h \rightarrow y$ (Substitution)

$x_0 \rightarrow y_0$. But, initial point is x so, $y_0 = x$

Write a Taylor series of $f(y)$ then convert to $f(x+h)$

$$f(y) = f(y_0) + f'(y_0)(y - y_0) + \frac{(y - y_0)^2}{2!} f''(y_0) + \dots$$

$$\Rightarrow f(x + h) \cong f(x) + f'(x)(x + h - x) + f''(x) \frac{h^2}{2!} + \dots$$

Taylor Series for $f(x+h)$

$$f(x + h) \cong f(x) + f'(x)(x + h - x) + f''(x) \frac{h^2}{2!} + \dots + f^{(n)}(x) \frac{h^n}{n!} + \dots$$

$$= f(x) + f'(x)(h) + f''(x) \frac{h^2}{2!} + \dots + f^{(n)}(x) \frac{h^n}{n!} + \dots$$

$$= \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} h^k$$

$$f(x + h) = f(x) + f'(x)(h) + f''(x) \frac{h^2}{2!} + \dots + f^{(n)}(x) \frac{h^n}{n!} + E_{n+1}$$

$$= \sum_{k=0}^n f^{(k)}(x) \frac{h^k}{k!} + E_{n+1}$$

where,

$$E_{n+1} = f^{(n+1)}(\xi) \frac{h^{n+1}}{(n+1)!} = O(h^{n+1}) \quad , \quad x \leq \xi \leq x + h$$

$$|E_{n+1}| \leq c |h|^{n+1}$$

Memorize

Exercise

Taylor Series for $f(x)$

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{(x - x_0)^2}{2!} f''(x_0) + \dots + \frac{(x - x_0)^n}{n!} f^{(n)}(x_0) + \dots$$

$$f(x) = \sum_{k=0}^{\infty} f^{(k)}(x_0) \cdot \frac{(x - x_0)^k}{k!}$$

Taylor Series for $f(x+h)$

$$f(x + h) = f(x) + f'(x)(h) + f''(x) \frac{h^2}{2!} + \dots + f^{(n)}(x) \frac{h^n}{n!} + \dots$$

$$f(x + h) = \sum_{k=0}^{\infty} f^{(k)}(x) \cdot \frac{h^k}{k!}$$

Also, read (APPENDIX B) for Multi-variate Taylor Series

Appendix A

Representation of Numbers on a Digital System

Numbers are represented in digital systems with a fixed number of bits, such as 16-bits, 32-bits MCU, 64-bit CPU. Due to the finite number of bits in representing numbers, there exists **Round-off Errors** in numerical solutions.

For example, the number $2/3$ cannot be represented exactly in a computer and it needs to be *chopped-off* and the last digits requires to be *rounded*. Also, a number may not be represented exactly due to the resolution with finite bits.

We will review how numbers are represented in a computer with binary digits to understand more about **the round-off errors in numerical methods**.

Representing numbers in IEEE-754

IEEE – 754

	Bit	Byte	Numbers
Integer	16	2	-32768~32767
Unsigned int	16	2	0~65535
32 bits single (float)	32	4	$\pm 3.4e-38$ ($\pm 3.4e-38$)
64 bits double	64	8	$\pm 1.8e308$ ($\pm 1.1e-308$)
Long double	80	10	

Window 64=bit

	Bit	Byte	Numbers
Integer	32	4	-2,147,483,648~ 2,147,483,647
Unsigned int	32	4	0~ 4,294,967,295.
float	32	4	$\pm 3.4e-38$ ($\pm 3.4e-38$)
double	64	8	$\pm 1.8e308$ ($\pm 1.1e-308$)

Floating Point Representation

A real number is expressed in floating point representation with a sign bit, exponent bits, and mantissa bits.

$$\begin{array}{c} \text{sig} \rightarrow \pm 0. d_1 d_2 \cdots \times 10^n \\ \text{Mantis} \end{array} \quad (\text{decimal})$$

$$\pm 1. d_1 d_2 \cdots \times 2^n \quad (\text{binary})$$

Float and Double Precision

	Sign	Exponent	Mantissa	
Float (Single precision)	1bit	8	23	32bits (4 bytes)
Double (Double precision)	1bit	11	52	64bits (8 bytes)

The value of the exponent is entered with a bias

$$\text{Floating point in binary : } (\pm 1) \times 2^{c-127} \times (1.f)_2$$

Biased exponent
Mantissa

$$(-1)^{b_1} \times 2^{(b_2 b_3 \cdots b_9)_2} \times 2^{-127} \times (1.b_{10} \cdots b_{32})_2$$

Q. How can you represent '0' or 'INF' with the above representation?

0 : 00000000

INF : 11111111

Number range by a float-point number

$$(-1)^{b_1} \times 2^{(b_2 b_3 \dots b_9)_2} \times 2^{-127} \times (1.b_{10} \dots b_{32})_2$$

Exponent range with bias: 8 bit

$$0 - 127 \leq c \leq 255 - 127 = 128$$

But, -127, 128 are reserved for '0', 'INF'.

Thus, the available range of exponent is -126~127

Fraction(mentisa) range: 23 bits

$$1 \leq (1.b_{10} \dots b_{32})_2 \leq 2 - 2^{-23} \approx 2$$

One word: 32 bits (4 bytes) of float range

$$2^{-126} \approx 1.2 \times 10^{-38} \leq x \leq 3.4 \times 10^{38} \approx 2^{127} \cdot 2 = 2^{128}$$

$$-2^{128} \approx -3.4 \times 10^{38} \leq x \leq -1.2 \times 10^{-38} \approx -2^{126}$$

Two words: 32+32 bits (double)

$$(-1)^{b_1} \times 2^{(b_1 b_2 \dots b_{12})_2} \times 2^{-1023} \times (1.b_{13} b_{14} \dots b_{64})_2$$

Smallest positive number close to '0'

- Float $2^{-126} \approx 1.2 \times 10^{-38}$
- Double $2^{-1022} \approx 2.2 \times 10^{-308}$

Precision(resolution):

The smallest gap between two number (ε) – smallest mantissa(the distance from 1.0 to the next largest number)

- Float $2^{-23} \approx 1.2 \times 10^{-7}$
- Double $2^{-52} \approx 2.2 \times 10^{-16}$

Q. How about the type of **integer & character?** (int x, char x, unsigned x) Check with your CPU/Compiler

- int : 4byte
- char : 1byte -128 to 127

Appendix B

More notes on Taylor Series Expansion

Special case when $n=0$

Taylor series of $f(x)$ when $n = 0$

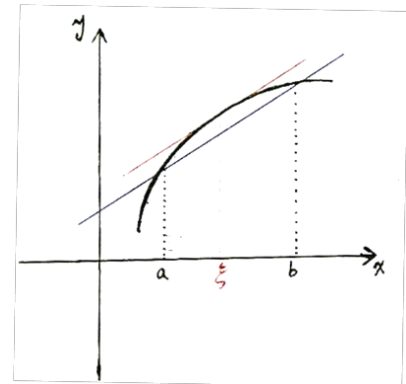
$$\begin{aligned} f(x) &= f(x_0) + E_1 \\ &= f(x_0) + f'(\xi)(x - x_0) \end{aligned}$$

Let $x = b$, $x_0 = a$

$$f(b) = f(a) + f'(\xi)(b - a)$$

$$\Leftrightarrow f'(\xi) = \frac{f(b) - f(a)}{b - a} \quad (\text{Mean value Theorem})$$

for some ξ between (a, b)



Taylor series for Multivariable function

$f(x, y)$ about (x_0, y_0)

$$\begin{aligned} f(x, y) &= f(x_0, y_0) + \left[\frac{\partial f(x_0, y_0)}{\partial x} + \frac{\partial f(x_0, y_0)}{\partial y} (y - y_0) \right] \\ &\quad + \frac{1}{2!} \left[\frac{\partial^2 f(x_0, y_0)}{\partial x^2} (x - x_0)^2 + \frac{\partial^2 f(x_0, y_0)}{\partial y^2} (y - y_0)^2 + \frac{\partial^2 f(x_0, y_0)}{\partial x \partial y} 2(x - x_0)(y - y_0) \right] \\ &\quad \vdots \\ &\quad + \frac{1}{n!} \left[\sum_{k=0}^n \frac{n!}{k!(n-k)!} \left\{ \frac{\partial^n f(x_0, y_0)}{\partial x^k \partial y^{n-k}} (x - x_0)^k (y - y_0)^{n-k} \right\} \right] + \dots \end{aligned}$$

- Multi variable Taylor series:

Let f be an infinitely differentiable function in some open neighborhood around $(x, y) = (a, b)$.

$$f(x, y) = f(a, b) + f_x(a, b)(x - a) + f_y(a, b)(y - b) + \frac{1}{2!} [f_{xx}(a, b)(x - a)^2 + 2f_{xy}(a, b)(x - a)(y - b) + f_{yy}(y - b)^2] + \dots$$

- A more compact form:

Let $\mathbf{x} = \langle x, y \rangle$ and let $\mathbf{a} = \langle a, b \rangle$. With this new vector notation, the Taylor series can be written as

$$f(\mathbf{x}) = f(\mathbf{a}) + [(\mathbf{x} - \mathbf{a}) \cdot \nabla f(\mathbf{a})] + [(\mathbf{x} - \mathbf{a}) \cdot (H(\mathbf{x}) \cdot (\mathbf{x} - \mathbf{a}))] + \dots$$

where H is the matrix of second derivatives, called the **Hessian matrix**

$$H(x, y) = \begin{bmatrix} f_{xx}(x, y) & f_{xy}(x, y) \\ f_{yx}(x, y) & f_{yy}(x, y) \end{bmatrix}$$